

```

#define MAX_EVENTS 1024 /*Max. number of events to process at one
go*/
#define LEN_NAME 16 /*Assuming that the length of the filename
won't exceed 16 bytes*/
#define EVENT_SIZE ( sizeof( struct inotify_event ) ) /*size of
one event*/
#define BUF_LEN ( MAX_EVENTS * ( EVENT_SIZE + LEN_NAME ) )
/*buffer to store the data of events*/

int main( int argc, char **argv )
{
    int length, i = 0, wd;
    int fd;
    char buffer[BUF_LEN];

    /* Initialize Inotify*/
    fd = inotify_init();
    if ( fd < 0 ) {
        perror( "Couldn't initialize inotify");
    }

    /* add watch to starting directory */
    wd = inotify_add_watch(fd, argv[1], IN_CREATE | IN_MODIFY |
IN_DELETE);

    if ( wd == -1)
    {
        printf("Couldn't add watch to %s\n",argv[1]);
    }
    else
    {
        printf("Watching:: %s\n",argv[1]);
    }

    /* do it forever*/
    while(1)
    {
        i = 0;
        length = read( fd, buffer, BUF_LEN);

        if ( length < 0 ) {
            perror( "read" );
        }

        while ( i < length ) {
            struct inotify_event *event = ( struct inotify_event * )
&buffer[ i ];
            if ( event->len ) {
                if ( event->mask & IN_CREATE ) {
                    if (event->mask & IN_ISDIR)
                        printf( "The directory %s was Created.\n", event->name
);
                }
                else
                    printf( "The file %s was Created with WD %d\n", event-

```

```

>name, event->wd );
            }

            if ( event->mask & IN_MODIFY ) {
                if (event->mask & IN_ISDIR)
                    printf( "The directory %s was modified.\n", event->name
);
                else
                    printf( "The file %s was modified with WD %d\n", event-
>name, event->wd );
            }

            if ( event->mask & IN_DELETE ) {
                if (event->mask & IN_ISDIR)
                    printf( "The directory %s was deleted.\n", event->name
);
                else
                    printf( "The file %s was deleted with WD %d\n", event-
>name, event->wd );
            }

            i += EVENT_SIZE + event->len;
        }
    }

    /* Clean up*/
    inotify_rm_watch( fd, wd );
    close( fd );

    return 0;
}

```

Shown below is the sample output of compiling and running the above program:

```

$ gcc listing1.c
$ ./a.out /home/gene/
Watching:: /home/gene/
.
.
The file .xsession-errors was modified with WD 1
The file .xsession-errors was modified with WD 1
The file .xsession-errors was modified with WD 1
The directory foobar was Created.

```

The first step that you need to perform in your application is the initialisation of the Inotify instance using *inotify_init()*, which returns a file descriptor that will be used to add watches and read event data in the rest of the application.

Event data is returned as a structure variable of type *struct inotify_event* (defined in *linux/inotify.h*), which is read into a buffer using the standard *read()* system call.